

Session 5 Speaker Script — COMPLETE WITH FULL TALKING POINTS

Retrieval-Augmented Generation (RAG)

Presenter: Dinesh Wadhvani | **Session:** 5 of 12 | **Duration:** ~60 minutes

Purpose: Ground AI in real documents—end hallucinations, add citations

Slide	Title	Time
1	Title Slide	~1 min
2	Today's Journey	~2 min
3	The Hallucination Problem	~5 min
4	What is RAG?	~5 min
5	How Retrieval Works	~8 min
6	Embeddings & Similarity	~8 min
7	Prompting with Context	~8 min
8	Real-World RAG Systems	~8 min
9	Citations & Verification	~5 min
10	Building RAG Pipelines	~5 min
11	Lab Overview	~3 min
12	Key Takeaways	~5 min
13	Session 6 Preview	~2 min
—	Lab Exercise	~12 min

01. TITLE SLIDE

~1 minute

WHAT'S ON SCREEN

Session 5 label (blue) • Title: 'Retrieval-Augmented Generation' • Subtitle: 'Grounding AI in Real Documents' • Session 5 of 12

SPEAKING SCRIPT

Welcome to Session 5. We've built a lot of capabilities. Session 2: you learned how LLMs work mechanically. Session 3: you learned to write prompts that work. Session 4: you learned to make LLMs generate structured data that systems can process.

Today we're solving one of the biggest problems with LLMs: hallucination. The tendency to confidently state false information. Because LLMs are pattern-matchers, not fact-checkers, they can be completely wrong while sounding completely right.

The solution: Retrieval-Augmented Generation. RAG. Instead of asking the model 'What do you know?' you ask it 'Given these documents, what can you answer?'

It's a simple idea with profound implications. Because suddenly your AI isn't just intelligent. It's verifiable. It cites sources. It's grounded in reality.

By the end of today, you'll understand how RAG works, you'll see real products doing this at scale, and you'll build a simple RAG system yourself. In the lab, you'll take your resume data from Session 4 and build a retrieval system that can answer questions about it—WITH citations.

This is where AI becomes trustworthy.

DELIVERY TIPS

High energy—this solves a real problem people complain about. • Start with a concrete hallucination example: 'Ask ChatGPT something made up and it'll confidently explain it.' • Emphasize: RAG isn't magic, it's practical grounding. • By end of session, you'll build something you could use in production.'

02. TODAY'S JOURNEY

~2 minutes

WHAT'S ON SCREEN

13-item agenda • Items: Hallucinations, RAG concept, Retrieval, Embeddings, Prompting, Real examples, Citations, Pipelines, Lab

SPEAKING SCRIPT

Quick roadmap. We start with five minutes on the hallucination problem. What is it? Why does it happen? Why should you care?

Then five minutes on what RAG actually is. Simple concept but powerful implications.

Eight minutes on how retrieval works. How do you find the right documents when you need them?

Eight minutes on embeddings and similarity. This is the mathematical trick that makes modern retrieval possible.

Eight minutes on prompting with retrieved context. You already know how to prompt. Now you add 'here are relevant documents, answer based on these.'

Eight minutes on real-world RAG systems. Legal tech, medical AI, customer support, enterprise search. You'll see this is already everywhere.

Five minutes on citations and verification. How do you make AI verifiable?

Five minutes on building RAG pipelines. From documents → retrieval → context → answer, end-to-end.

Then lab—12 minutes. You'll build a simple but real RAG system with your resume data. Questions go in, answers with citations come out.

Then takeaways and Session 6 preview. If time gets tight, we compress the embeddings section. We protect retrieval, RAG concept, and lab.

DELIVERY TIPS

Move briskly—this is a foundational session. • Emphasize that RAG is the foundation for trustworthy AI. • Preview the lab: 'You'll build something that could be used in real hiring systems.'

03. THE HALLUCINATION PROBLEM

~5 minutes

WHAT'S ON SCREEN

Problem-Consequence visual • LLM confidently wrong examples • Real business costs • Why it happens (from Session 2 mechanics)

SPEAKING SCRIPT

Let me start with the problem. Ask ChatGPT: 'What is the tallest building in Mumbai?' It will confidently give you an answer. Sounds authoritative. Well-explained. Probably sounds plausible.

But here's the thing: ChatGPT has no idea. It doesn't have access to current information. It's pattern-matching based on training data. If the training data said X is the tallest, it'll say X. If training data is conflicting, it might say whatever pattern is strongest. It's not checking reality. It's predicting what would statistically come next.

This is called hallucination. Not lying intentionally. But confidently generating false information.

WHY IT HAPPENS (MECHANICS FROM SESSION 2):

Remember: LLMs work by predicting the next token. If you ask a question, the model predicts: what words would follow this question? If the training data has answers, it predicts those tokens. But if the training data has wrong answers, or conflicting answers, or no answers, the model still has to predict SOMETHING. And it predicts statistically likely text, not necessarily true text.

REAL-WORLD CONSEQUENCES:

A lawyer uses ChatGPT to research case law. Gets back citations that sound real but don't exist. Cites them in a legal brief. Judge catches it. Lawyer looks incompetent. Law firm faces sanctions.

A doctor uses AI to check symptoms. AI confidently recommends wrong treatment because it hallucinated about that symptom combination. Patient gets wrong treatment. Real harm.

A company uses AI for customer support. AI confidently gives wrong information about a product. Customer is frustrated. Brand trust damaged.

A researcher uses AI to summarize papers. AI cites studies that don't exist or misrepresents them. Researcher publishes based on hallucinations. Paper is retracted. Reputation damaged.

BUSINESS IMPACT:

Companies are spending millions on AI. But if they can't trust the output, it's worthless or worse—actively harmful.

THIS IS WHERE RAG COMES IN.

Solution: Instead of 'answer based on what you learned in training,' say 'answer based on these specific documents I'm giving you.'

Now the AI can't hallucinate about things not in the documents. It's grounded. It's verifiable. You can check its work.

DELIVERY TIPS

Real consequences make this visceral. Use the lawyer example—students understand high stakes. • Medical and legal examples show this isn't theoretical. • The mechanic explanation (from Session 2) grounds it: students understand why it happens. • Build anticipation: 'There IS a solution.'

04. WHAT IS RAG?

~5 minutes

WHAT'S ON SCREEN

RAG concept diagram • Input question → Retrieve documents → Add to prompt → LLM answers → Output with source • Before/After comparison

SPEAKING SCRIPT

RAG stands for Retrieval-Augmented Generation. Let me break that down.

Retrieval: You have a collection of documents. When a question comes in, you search for the most relevant documents.

Augmented: You 'augment' your prompt by including those retrieved documents as context.

Generation: The LLM generates an answer based on the augmented prompt (question + context).

SIMPLE EXAMPLE:

WITHOUT RAG:

Question: 'What was Priya's most recent job?'

Model thinks: 'I don't have specific information about Priya in my training data. I'll guess based on common patterns.'

Answer: 'Probably something in tech. Maybe a data analyst role. I'm not sure.'

Problem: Uncertain, possibly wrong, no source.

WITH RAG:

Question: 'What was Priya's most recent job?'

System retrieves: [Priya's resume from your documents] → found section: 'Experience: Most recent position is Senior Data Analyst at TechCorp, 2 years'

Model now sees: 'Question: What was Priya's most recent job? Context from resume: [most recent experience details]'

Answer: 'According to the resume, Priya's most recent job was Senior Data Analyst at TechCorp.'

Benefit: Certain, correct, cited source.

THE POWER:

By giving the model specific documents, you constrain what it can say. It's not inventing. It's extracting and summarizing what's actually there.

THREE COMPONENTS OF RAG:

1. KNOWLEDGE BASE: Your documents. Could be resumes, case law, medical journals, product documentation, anything.

2. RETRIEVAL SYSTEM: When a question comes in, find the most relevant documents. Fast and accurate retrieval is key.

3. PROMPTING STRATEGY: Take the question + retrieved documents + ask the model to answer based on these sources. The LLM's job is now narrower: 'Answer based on what's here,' not 'answer from all of human knowledge.'

WHY THIS WORKS:

You're moving from: LLM must know everything → LLM must intelligently use provided information

Much easier. Much more reliable. Much more verifiable.

DELIVERY TIPS

The before/after comparison is powerful. Show the difference clearly. • Simple example (Priya's job) is concrete and relatable. • Three components breakdown is essential—students need to understand RAG isn't magic, it's systematic. • Connection: 'This is the next level of prompt engineering—you're now providing context strategically.'

05. HOW RETRIEVAL WORKS

~8 minutes

WHAT'S ON SCREEN

Retrieval pipeline visual • Question → Search → Rank → Return top K • Three retrieval methods shown: keyword, semantic, hybrid

SPEAKING SCRIPT

Now let's talk about retrieval. How do you find relevant documents quickly?

SIMPLE APPROACH: KEYWORD SEARCH

Most basic: search for documents containing the question's keywords.

Question: 'What are Priya's technical skills?'

Search for documents containing: 'Priya' AND 'technical' AND 'skills'

Return matching documents

Pro: Fast, simple, works for exact matches

Con: Misses semantic relationships. If document says 'programming languages' instead of 'technical skills', keyword search misses it.

Example failure: Search for 'salary' but document says 'compensation'. Keyword search returns nothing, but they mean the same thing.

BETTER APPROACH: SEMANTIC SEARCH

Instead of keywords, you search by meaning. 'What's the semantic closest document to this question?'

This uses something called embeddings (we'll get into details in next slide). Short version: you convert documents and questions into numerical representations. Then find documents closest to the question numerically.

Question: 'What are Priya's technical skills?'

System converts to embedding (mathematical representation)

Searches for documents with similar embeddings

Finds: documents talking about 'programming languages', 'technical expertise', 'coding proficiency' even if they don't use exact phrase 'technical skills'

Pro: Understands meaning, finds related concepts, flexible

Con: More complex, requires pre-computing embeddings

HYBRID APPROACH: KEYWORD + SEMANTIC

Combine both. Search with keywords AND semantic similarity. Return best results from both approaches.

Example: Question 'What companies did Priya work for?'

Keyword search: finds documents with 'company' or 'work'

Semantic search: finds documents talking about 'employer' or 'organization' or 'job'

Combine results. Return union of both.

Most practical systems use hybrid because you get precision (keyword) + recall (semantic).

RANKING AND RELEVANCE:

When you get back multiple documents, rank them by relevance. Most relevant first.

For keyword search: count keyword matches. More matches = higher rank.

For semantic search: measure distance to question embedding. Closer = more relevant.

THE RETRIEVAL PIPELINE:

1. Question comes in
2. Search (keyword, semantic, or both)
3. Get candidate documents
4. Rank by relevance
5. Return top K (usually top 3-5)
6. Pass to LLM as context

REAL-WORLD CONSIDERATIONS:

Speed matters. If retrieval takes 10 seconds, users wait 10 seconds. Systems typically need retrieval in milliseconds or a few seconds max.

Accuracy matters. If you retrieve the wrong documents, the LLM works with wrong context. Bad output follows.

This is why companies invest heavily in retrieval engineering. Getting retrieval right is often harder than getting the LLM right.

DELIVERY TIPS

Retrieval is the 'plumbing' of RAG—less flashy than LLM but equally important. • Three approaches show progression: simple → better → practical. • Speed vs accuracy tradeoff is real and practical. • Students should understand: RAG is only as good as retrieval. Garbage in, garbage out.

06. EMBEDDINGS & SIMILARITY

~8 minutes

WHAT'S ON SCREEN

Embedding concept visual • Text → Vector representation • Similarity calculation shown • 2D visualization of similar documents close together

SPEAKING SCRIPT

Here's the mathematical trick that makes modern semantic search work: embeddings.

AN EMBEDDING IS A NUMERICAL REPRESENTATION OF TEXT.

Text is human-readable: 'Priya is a data scientist with 3 years of experience.'

Embedding is mathematical: [0.25, -0.81, 0.42, ..., -0.19] (imagine 768 or 1,536 numbers like this)

The embedding captures meaning. Words with similar meanings have similar embeddings. 'Data scientist' and 'machine learning engineer' have similar embeddings. 'Engineer' and 'student' have different embeddings.

HOW EMBEDDINGS ARE CREATED:

Companies have trained embedding models (same way LLMs are trained, but for a different task). Given text, these models output embeddings.

Example: OpenAI has text-embedding-3-small. Google has Gecko. These are embedding models.

You pass text through: 'Priya is a data scientist' → embedding model → [numerical vector]

WHY EMBEDDINGS MATTER FOR RAG:

Because you can measure similarity between embeddings using math.

Simple example: cosine similarity. Two embeddings are similar if the angle between them is small. This gives you a number between -1 and 1 (usually 0 to 1 in practice). Higher = more similar.

Application to retrieval:

1. Create embedding for the question: 'What are Priya's skills?'
2. Create embeddings for all documents in your knowledge base (done once, stored)
3. Compute cosine similarity between question embedding and each document embedding
4. Sort by similarity score
5. Return documents with highest scores

PRACTICAL EXAMPLE:

Question: 'What companies has Priya worked at?'

Question embedding: [...some vector...]

Document 1 (Priya's experience section): [...vector A...] → similarity: 0.92 ✓ Very similar

Document 2 (Priya's skills section): [...vector B...] → similarity: 0.45 ✓ Somewhat similar

Document 3 (Unrelated person's info): [...vector C...] → similarity: 0.12 ✗ Not similar

Document 4 (Wikipedia on companies): [...vector D...] → similarity: 0.10 ✗ Not similar

Retrieval returns: Document 1 (most relevant)

VECTOR DATABASES:

Storing and searching embeddings efficiently requires special databases optimized for vector operations.

Examples: Pinecone, Weaviate, Milvus, FAISS

These databases let you:

- Store vectors efficiently
- Search millions of vectors in milliseconds
- Scale from thousands to billions of documents

REAL-WORLD SCALE:

LinkedIn stores embeddings for millions of resumes. When you search for a job, they use semantic search on resume embeddings to find matches. Millions of documents searched in milliseconds.

Google's semantic search on web results uses embeddings at massive scale.

WHY EMBEDDINGS ARE REVOLUTIONARY:

Before embeddings, you had to rely on keywords. Limited, fragile.

With embeddings, you can search by meaning. 'Resume' vs 'CV' vs 'professional profile' all understood as similar concepts.

This is why modern retrieval is so much better than older systems.

DELIVERY TIPS

Embeddings are mathematical but explanation should be intuitive. Use the angle/distance metaphor. • Real companies (LinkedIn, Google) using this at scale shows importance. • The practical example (similarity scores) makes it concrete. • Vector databases are mentioned so students know these tools exist—they'll encounter them in real work. • Connection: 'This is how modern systems solve the problem that keyword search couldn't.'

07. PROMPTING WITH CONTEXT

~8 minutes

WHAT'S ON SCREEN

Prompt template for RAG • Retrieved documents shown, then question • Answer grounded in context • Citations format shown

SPEAKING SCRIPT

Now you know how retrieval works. Let's talk about prompting the LLM with retrieved context.

TEMPLATE FOR RAG PROMPTS:

Use this structure:

'You are a helpful assistant. Answer the question BASED ONLY on the provided context. If the answer is not in the context, say "I don't have that information." Do not rely on prior knowledge. Always cite your source.'

CONTEXT:

[Retrieved documents go here]

QUESTION: [User's question]

ANSWER:'

KEY ELEMENTS:

1. RESTRICTION: 'Answer based ONLY on provided context' - this prevents hallucination. Model must stay within bounds.
2. FALLBACK: 'If not in context, say you don't know' - better to admit ignorance than hallucinate.
3. CITATION: 'Always cite your source' - makes output verifiable.

EXAMPLE: RESUME RAG

Context (retrieved from resume):

'Experience: Senior Data Analyst at TechCorp (2021-2023), 2 years. Responsibilities: Built dashboards, optimized SQL queries, trained junior analysts. Achievements: Reduced query time by 40%, created automated reporting system.'

Question: 'What was Priya's most significant achievement?'

Model's answer (with context): 'Based on the resume, Priya's most significant achievement was reducing query time by 40% (Source: Experience section, TechCorp role). She also created an automated reporting system.'

Compare to without context: 'I'm not sure, but probably something related to data analysis or leadership since that seems common for career progression.' (Vague, unverifiable)

PROMPTING TECHNIQUES FOR RAG:

TECHNIQUE 1: STRICT GROUNDING

'Answer based ONLY on provided text. Do not add information from your training.'

Use when: You need 100% accuracy, legal/medical context, or untrusted sources.

TECHNIQUE 2: BALANCED REASONING

'Primary source is the provided context. If context is unclear, you may use general knowledge, but always cite whether answer comes from context or general knowledge.'

Use when: Context might be incomplete, supplementary info helpful, but primary should be context.

TECHNIQUE 3: INTERACTIVE CLARIFICATION

'If the provided context is insufficient to answer the question, ask clarifying questions about what specific information the user needs.'

Use when: Question is ambiguous or context might be incomplete.

FOR MOST CASES, USE STRICT GROUNDING:

It's the safest. Prevents hallucination. Students should start here.

HANDLING CITATIONS:

When model cites sources, make sure they're meaningful. Examples:

Good citation: 'According to the Experience section, TechCorp role (Source: resume line 15-20)'

Bad citation: 'In the context provided (Source: context)' (too vague)

Your prompt should encourage specific citations. Example:

'When citing, include: the specific section name, the role or company mentioned, and the time period if relevant.'

REAL-WORLD EXAMPLE: LEGAL AI

System retrieves relevant case law from database.

Prompt: 'Based on the retrieved case law below, does this situation have precedent? Answer based ONLY on the cases provided. Cite specific case names and rulings.'■r/>

Retrieved cases: [Smith v. Jones, 2019 - ruling on X], [Brown v. State, 2021 - ruling on Y]

Model's response: 'Yes, there is precedent. In Smith v. Jones (2019), the court ruled that [specific ruling]. This applies to your situation because [how it applies].'

Lawyer can now verify by looking up Smith v. Jones. Output is reliable.

CONTRAST WITH PURE LLM:

Without RAG (pure LLM): 'There might be precedent similar to this. Cases like Jones v. Smith might be relevant. You should research...'

Problem: Vague, possible hallucination, no verification possible.

With RAG: 'Yes, precedent exists in Smith v. Jones (2019) where the court ruled [specific].'

Benefit: Specific, verifiable, citable.

DELIVERY TIPS

The template is immediately practical—students will use it. • Real examples (resume, legal) show applications. • Emphasis on strict grounding at start—safety first. • Citations are crucial for trustworthiness—make this clear. • Show the contrast before/after to motivate why RAG works better.

08. REAL-WORLD RAG SYSTEMS

~8 minutes

WHAT'S ON SCREEN

5 Real products/companies using RAG • Bing Chat, GitHub Copilot, LegalTech, MedAI, Enterprise Search • How each uses RAG • Business benefits

SPEAKING SCRIPT

Let me show you real products using RAG. This is already everywhere.

PRODUCT 1: BING CHAT (Microsoft)

What it does: You ask a question. Bing searches the web, retrieves relevant pages, feeds them to the LLM, and generates an answer with citations.

Why RAG: Without retrieval, Bing Chat couldn't answer current questions (LLM training data is old). With RAG, it retrieves current web pages and grounds answers in today's reality.

User experience: 'What are the latest stock market trends?' → Bing retrieves today's financial news → LLM summarizes based on news → Output includes links to sources.

Business impact: Users trust the results because they can verify by checking sources. Accuracy goes up dramatically vs pure LLM.

PRODUCT 2: GITHUB COPILOT WITH DOCUMENTATION

What it does: When you ask for code help, Copilot can retrieve relevant documentation, Stack Overflow answers, examples from the codebase, then generate code based on these sources.

Why RAG: Instead of hallucinating function names or APIs, Copilot retrieves actual documentation and generates code that matches actual library APIs.

Example: 'How do I connect to a PostgreSQL database in Node.js?' → Retrieves PostgreSQL Node library docs → Retrieves examples from GitHub → Generates code based on real API, not hallucinations.

Business impact: Code quality improves. Fewer bugs from using wrong function signatures. Developer confidence in AI-generated code goes up.

PRODUCT 3: LEGAL TECH (LexisNexis, Thomson Reuters)

What it does: Lawyers ask questions about case law. System retrieves relevant cases from database, feeds to LLM, generates legal analysis with case citations.

Why RAG: Legal precedent must be accurate and citable. Without RAG, LLM might cite cases that don't exist or misrepresent rulings. With RAG, only real cases are used, and citations are verifiable.

Example: 'What's the precedent on contract interpretation in ambiguous clauses?' → Retrieves 10 most relevant cases from the database → LLM synthesizes and cites them → Output is a legal analysis with reliable sources.

Business impact: Reduces time lawyers spend researching (from hours to minutes). Increases confidence in citations (legally defensible). High-value service.

PRODUCT 4: MEDICAL AI (PubMed, clinical decision support)

What it does: Doctors ask clinical questions. System retrieves peer-reviewed medical literature, feeds to LLM, generates clinical guidance with citations to actual studies.

Why RAG: Medical advice must be based on evidence. Without RAG, LLM might hallucinate treatments or misremember studies. With RAG, only real studies are cited and can be verified.

Example: 'What's the evidence for treatment X for condition Y?' → Retrieves recent studies from PubMed → LLM summarizes findings → Output includes citations to actual research.

Business impact: Doctors trust the tool more because it's grounded in evidence. Reduces liability (citations to published research provide defense). Regulatory bodies like FDA are more comfortable with RAG-based medical AI.

PRODUCT 5: ENTERPRISE SEARCH (Slack, Notion)

What it does: Employees search internal knowledge base. Instead of just returning documents, system uses RAG to generate a direct answer to the question, citing internal sources.

Why RAG: Instead of 'Search found these 5 documents, go read them,' the system provides 'The answer to your question is X (found in Finance Handbook page 3 and HR Policy doc 7).'

Example: 'What's the vacation policy?' → Retrieves relevant HR docs → LLM generates answer → Output: 'You get 20 days of vacation annually, plus 10 company holidays (Source: HR Handbook v2.3)'

Business impact: Employees get answers faster. Reduces questions to support teams. Improves knowledge discoverability.

COMMON PATTERN:

Notice what's consistent across all five: Question → Retrieve relevant source material → Generate answer based on sources → Cite sources → User can verify.

This is why RAG is spreading. It makes AI more trustworthy by making it verifiable.

MARKET GROWTH:

Companies are investing billions in RAG infrastructure. Vector databases (Pinecone, Weaviate) are exploding. Every major LLM company (OpenAI, Google, Anthropic) is building RAG capabilities.

Why? Because RAG solves the trust problem. And trust is what separates toy AI from production AI.

DELIVERY TIPS

Five real examples from companies students have heard of. GitHub Copilot and Bing Chat are especially relatable. • Each example shows: problem → RAG solution → business benefit. • Pattern at the end ties them together. • Emphasize: this is already mainstream, not experimental. • Market growth shows this is where the future is headed.

09. CITATIONS & VERIFICATION

~5 minutes

WHAT'S ON SCREEN

Citation strategies shown • Direct quotes vs paraphrasing • Source attribution • Verification checklist

SPEAKING SCRIPT

An important part of RAG: citations make output verifiable. Let me cover citation best practices.

WHY CITATIONS MATTER:

Citations do two things: 1) Give credit to sources, 2) Allow verification. If someone reads an AI's answer and wants to verify, they can look at the source.

TWO CITATION APPROACHES:

APPROACH 1: DIRECT QUOTES

'According to the resume, "Senior Data Analyst at TechCorp (2021-2023)." (Source: Resume, Experience section)'

Advantage: Exact, no interpretation by AI

Disadvantage: Reads awkwardly sometimes, context might be missing

Use when: Specific facts, numbers, dates where accuracy is critical

APPROACH 2: PARAPHRASING WITH ATTRIBUTION

'Priya worked as a Senior Data Analyst at TechCorp for two years. (Source: Resume, Experience section)'

Advantage: More natural language, reads better

Disadvantage: AI might subtly misrepresent; less verifiable than direct quote

Use when: Summarizing information, general concepts where slight paraphrasing is okay

BEST PRACTICE: HYBRID

Use direct quotes for key facts. Paraphrase for background. Always cite source.

'Priya was a Senior Data Analyst at TechCorp for two years. Key achievement: "Reduced query time by 40% and created an automated reporting system." (Source: Resume, Experience section)'

CITATION SPECIFICITY:

Vague citation: 'Source: Resume' (which part?)

Better: 'Source: Resume, Experience section, TechCorp role'■r/>Best: 'Source: Resume, Experience section, TechCorp role (2021-2023, line 12-14)'■r/>

More specific = more verifiable = more trustworthy

VERIFICATION CHECKLIST:

When you read an AI answer with citations, check:

- Does the citation exist? (Can you find it?)
- Does the citation support the claim? (Does it actually say that?)
- Is the interpretation accurate? (Did AI paraphrase correctly?)
- Is context preserved? (Did AI leave out important nuance?)
- Are there contradictory citations? (Did AI pick cherry-picked source?)

HANDLING WHEN SOURCES CONFLICT:

If retrieved documents conflict: 'Document A says X, Document B says Y.'

Good response: 'There are conflicting sources. Document A (resume) states [X], while Document B (interview notes) states [Y]. The most recent source is [Document C] which says [Z]. (Sources: Resume vs Interview notes vs Recent email)'

Transparency about conflicts builds trust.

BAD PRACTICE: HALLUCINATED CITATIONS

Never do this: 'According to Smith et al. (2019)...' when Smith et al. 2019 doesn't exist in your knowledge base.

Even worse than no citation. Actively deceptive.

Always retrieve before citing. Only cite what's actually in your knowledge base.

DELIVERY TIPS

Citations are about trust. Emphasize this repeatedly. • Hybrid approach (quotes for facts, paraphrase for background) is practical best practice. • Verification checklist is powerful—shows students what to look for when evaluating AI output. • Conflicting sources handling is mature and real—shows you're thinking about complexity. • The 'never hallucinate citations' rule is critical—must be drilled in.

10. BUILDING RAG PIPELINES

~5 minutes

WHAT'S ON SCREEN

RAG pipeline architecture • Document ingestion → Processing → Storage → Retrieval → LLM → Output • Each step shown with technology examples

SPEAKING SCRIPT

Let's talk about building RAG systems end-to-end. How do you go from 'I have documents' to 'I can answer questions about them'?

RAG PIPELINE ARCHITECTURE:

STEP 1: DOCUMENT INGESTION

Get documents into the system. Could be PDFs, text files, web pages, database records, anything.

Tools: Python libraries, scrapers, API connectors, file upload systems

STEP 2: DOCUMENT PROCESSING

Clean and structure the documents. Remove formatting noise, split into chunks if large, normalize text.

Tools: LangChain, Unstructured (document parsing library)

STEP 3: EMBEDDING GENERATION

Convert each document chunk to embeddings. Happens once, stored in database.

Tools: OpenAI embeddings API, Google Gecko, open-source models

Cost: Depends on document volume. For 1000 documents, might be \$1-10

STEP 4: VECTOR STORAGE

Store embeddings in a vector database for fast similarity search.

Tools: Pinecone, Weaviate, Milvus, FAISS (open source)

STEP 5: RETRIEVAL AT QUERY TIME

When user asks a question: embed the question, search vector database for similar documents, return top K

Speed: Should be fast, typically <1 second

STEP 6: PROMPT CONSTRUCTION

Take question + retrieved documents, construct prompt for LLM

Tools: Prompting libraries, template engines

STEP 7: GENERATION

LLM generates answer based on question + context

Tools: OpenAI API, Anthropic API, open-source models

STEP 8: POST-PROCESSING

Extract citations, format response, validate output

Tools: Custom code, regex, parsing libraries

REAL-WORLD COMPLEXITY:

Simple RAG: Steps 1-8 sequentially, works for small domains

Production RAG: Monitoring (track retrieval quality), caching (store frequent queries), feedback loops (users say 'this was wrong' → improve retrieval), multi-stage retrieval (coarse search → fine search), re-ranking (retrieved documents sorted by relevance before passing to LLM)

COST CONSIDERATION:

Each component has cost:

- Embeddings API: \$0.02 per 1M tokens (cheap)
- Vector database: \$0 (open source) to \$100+/month (managed)

- LLM API: \$0.01-0.10 per 1K tokens (depends on model)
- Hosting: \$50-1000+/month depending on scale

For small use case (10,000 documents, 1000 queries/month): ~\$50-200/month

For large scale (millions of documents, millions of queries): millions/month (but highly profitable)

IN TODAY'S LAB:

You'll build simplified version: document chunks → manual search (or simple keyword) → LLM answer → citations
Same principles, smaller scale.

DELIVERY TIPS

Pipeline architecture shows that RAG is systematic, not magic. • Technology names (Pinecone, LangChain) let students know what tools exist. • Cost breakdowns are realistic and practical. • Simple vs production RAG distinction shows this scales. • Lab connection: 'You're building the real system, just simplified.'

11. LAB EXERCISE OVERVIEW

~3 minutes

WHAT'S ON SCREEN

Lab title: 'Build Your Resume RAG System' • Step-by-step: Create knowledge base, retrieve on question, generate answer with citation • Deliverables

SPEAKING SCRIPT

Alright, lab time. You're going to build a functioning RAG system. Not theoretical. Real.

YOUR TASK: Use your Session 4 resume data (structured JSON). Create a retrieval system. Answer questions about the resume WITH citations.

STEP 1: CREATE KNOWLEDGE BASE (2 min)

Your 'documents' are the resume sections from Session 4: experience, skills, education.

STEP 2: CREATE RETRIEVAL (3 min)

When asked a question, find the most relevant resume section. Could be keyword matching or simple semantic (doesn't need to be sophisticated).

STEP 3: PROMPT FOR ANSWER (2 min)

Take question + retrieved section. Prompt LLM: 'Based on this resume section, answer this question. Cite the source.'

STEP 4: GENERATE WITH CITATION (2 min)

LLM answers, citing the resume section it used.

STEP 5: TEST & COMPARE (2 min)

Ask 3 questions. See RAG answer vs non-RAG answer. Notice the difference.

DELIVERABLES:

1. Resume sections formatted as 'knowledge base'
2. Retrieval mechanism (even if simple keyword search)
3. Test questions and answers (with RAG)
4. Comparison: RAG vs non-RAG
5. Reflection: How would this scale?

This is real. By end of lab, you'll have built something LinkedIn, Bing, and GitHub use.

DELIVERY TIPS

Step-by-step makes it achievable. • Emphasize: this is the real architecture, just simplified. • RAG vs non-RAG comparison is the powerful learning moment. • Students build something production-like.

12. KEY TAKEAWAYS FROM SESSION 5

~5 minutes

WHAT'S ON SCREEN

6 Core Principles • Hallucinations, RAG solution, Retrieval, Embeddings, Citations, Verification

SPEAKING SCRIPT

Takeaway One: LLMs Hallucinate—This is Fundamental

They're pattern-matchers, not fact-checkers. Confident but often wrong. This isn't a bug, it's inherent to how they work.

Takeaway Two: RAG Solves This—By Grounding

Give the model specific documents. Answer based on these. Suddenly, no hallucination possible (or at least bounded).

Takeaway Three: Retrieval is the Hard Part

Generating answers is easy for LLMs. Finding the RIGHT documents to generate from is hard. Good retrieval is engineering work.

Takeaway Four: Embeddings Power Modern Retrieval

Converting text to vectors lets you search by meaning, not keywords. This is why modern systems are so much better.

Takeaway Five: Citations Make Output Verifiable

The whole point of RAG is trustworthiness. Citations back that up. You can verify.

Takeaway Six: RAG is Everywhere Now

Bing, GitHub, legal tech, medical AI, enterprise search. This is how production AI works.

DELIVERY TIPS

These six principles are foundational. • Each takeaway has both concept and practical benefit. • Emphasize that RAG isn't future—it's present. Every serious AI application uses it.

13. COMING UP NEXT

~2 minutes

WHAT'S ON SCREEN

Dark blue background • Title: 'Session 6: Fine-Tuning & Specialization' • Preview bullets • Closing remarks

SPEAKING SCRIPT

Session 6 is where you customize LLMs for your specific domain. RAG grounds output in documents. Fine-tuning teaches the model your domain.

Combine them and you have AI that's both trustworthy AND specialized.

Difference: RAG says 'answer based on these documents.' Fine-tuning says 'you learn from examples of how to answer in this domain.'

Example: A customer service bot. RAG feeds it your knowledge base. Fine-tuning teaches it your company's tone, policies, and reasoning patterns.

In Session 6, you'll learn why fine-tuning works, when to do it (and when not to), and how to evaluate whether your fine-tuned model is actually better.

Thank you all for today. RAG is foundational. Master this and you can build trustworthy AI systems. See you in Session 6.

DELIVERY TIPS

Keep brief and energizing. • Session 6 natural progression: RAG (ground in documents) → Fine-tuning (teach the model). • End on note that combines RAG + fine-tuning for powerful systems.